

Q18. Application: Intelligent Search Optimization System A recursive search optimization algorithm reduces the input size by half and performs logarithmic work at each stage because of indexed search operations. Apply the substitution method to solve the recurrence $T(n)=T(n/2)+\log n$. Show all intermediate expansions and derive the final asymptotic time complexity. Analyze how the logarithmic work affects efficiency for large inputs. Interpret the result using asymptotic notation.

Experiment No.: Q18

Application: Intelligent Search Optimization System

Recurrence Relation: $T(n) = T(n/2) + \log_2 n$

Method Used: Substitution Method

Student Details

Name : T. KODANDA RAM

Register Number : 192524162

Department : AI&DS

Course : CSA0619-Design and Analysis of Algorithms

Faculty : Dr. F. Mary Harin Fernandez

Date : 17/07/2026

Aim

To implement an **Intelligent Search Optimization System** based on the recurrence relation $T(n)=T(n/2)+\log_2 n$, solve the recurrence using the **Substitution Method**, derive the **asymptotic time complexity**, and analyze how logarithmic work performed at each recursive stage affects the efficiency of the algorithm for large input sizes.

Objectives

The objectives of this experiment are:

- To understand the concept of recursive recurrence relations.
 - To study divide-and-conquer based search optimization.
 - To implement recursive algorithms using Python.
 - To understand indexed search operations.
 - To demonstrate recursive problem-size reduction.
 - To apply the substitution method for solving recurrences.
 - To derive the asymptotic time complexity mathematically.
 - To analyze the performance of logarithmic algorithms.
 - To compare logarithmic growth with linear growth.
 - To understand the practical applications of recursive search optimization systems.
-

Problem Statement

An Intelligent Search Optimization System repeatedly reduces the search space by half during every recursive call. At every stage, indexed search operations require logarithmic work to process the remaining search space.

The recurrence relation describing the algorithm is

$$T(n)=T(n/2)+\log_2 n$$

The objective is to

- recursively implement the recurrence,
 - apply the substitution method,
 - show all intermediate expansions,
 - derive the asymptotic time complexity,
 - analyze how logarithmic work affects the overall efficiency,
 - interpret the final complexity using asymptotic notation.
-

Introduction

Searching is one of the most fundamental operations in computer science. Every software application performs searching in one form or another. Whether searching customer records, locating files in a computer, querying a database, retrieving web pages from search engines, or processing large-scale cloud storage systems, efficient searching significantly improves overall system performance.

A naïve searching technique, such as linear search, examines every element sequentially and therefore requires linear time. As the amount of data increases, linear search becomes increasingly inefficient.

Modern search systems overcome this limitation using divide-and-conquer strategies. Instead of processing the complete dataset repeatedly, they recursively divide the problem into smaller parts, reducing the amount of work performed during each recursive stage.

The Intelligent Search Optimization System follows this approach. The search space is reduced by half after every recursive call. At each level, indexed search operations perform logarithmic work to identify the next region of interest. This significantly decreases the total computational effort compared to linear search algorithms.

The behavior of the algorithm can be represented mathematically using the recurrence relation

$$T(n) = T(n/2) + \log_2 n$$

where

- $T(n/2)$ represents solving the reduced problem.
- $\log_2 n$ represents the indexed search performed at the current level.

The objective of this experiment is to analyze this recurrence relation using the substitution method and derive its asymptotic complexity.

Theory

Recurrence Relation

A recurrence relation expresses the running time of a recursive algorithm in terms of the running time of smaller instances of the same problem.

Instead of writing the total running time directly, recurrence relations describe how the running time changes after each recursive call.

For example,

Binary Search

$$T(n) = T(n/2) + 1$$

Merge Sort

$$T(n) = 2T(n/2) + n$$

Quick Sort (Average)

$$T(n) = 2T(n/2) + n$$

In this experiment,

$$T(n) = T(n/2) + \log_2 n$$

This recurrence differs because the amount of work performed during each recursive level is logarithmic rather than constant or linear.

Recursive Problem Size Reduction

At every recursive call,

Input Size

n

↓

$n/2$

↓

$n/4$

↓

$n/8$

↓

$n/16$

↓

...

↓

1

Since the input size becomes half during every recursive stage, only **$\log_2 n$** recursive calls are required before reaching the base case.

For example,

If

$n=16$

the recursive calls are

16

↓

8

↓

4

↓

2

↓

1

Only four recursive reductions are required.

Indexed Search Operations

After reducing the problem size, the algorithm performs indexed search operations.

Indexed search avoids examining every element individually.

Instead, it directly accesses the required region through an index.

Examples include

- Binary Search
- B-Tree Search
- Database Index Search
- Hash Index Lookup
- Search Engine Index Traversal

These operations generally require logarithmic work.

Therefore,

Work at every recursive level = $\log_2 n$

Meaning of the Recurrence Relation

Given

$$T(n) = T(n/2) + \log_2 n$$

The first part

$$T(n/2)$$

represents the recursive solution.

The second part

$$\log_2 n$$

represents the indexed search work.

Therefore,

Total Running Time = Recursive Work

- Indexed Search Work

Intelligent Search Optimization System

An Intelligent Search Optimization System is designed to minimize the total amount of searching performed while maximizing efficiency.

Instead of searching every element, the system intelligently reduces the search space.

The algorithm performs three major operations.

Step 1

Reduce the search space by half.

Step 2

Perform indexed search.

Step 3

Repeat recursively until one element remains.

This approach dramatically improves performance for large datasets.

Real-World Applications

The recurrence relation used in this experiment appears in many practical systems.

Examples include

Database Systems

Databases maintain indexes for millions of records.

Searching is performed using index trees instead of sequential scanning.

Search Engines

Search engines maintain billions of indexed webpages.

Instead of checking every webpage, the search index quickly narrows the search space.

File Systems

Operating systems maintain indexed file tables.

Searching files requires logarithmic operations rather than linear traversal.

Artificial Intelligence

Many AI search algorithms repeatedly reduce the search space before selecting promising states.

Cloud Computing

Cloud storage systems use hierarchical indexing for rapid data retrieval.

Assumptions

The mathematical proof assumes

$$n=2^k$$

where

$$k=\log_2 n$$

This assumption simplifies the recurrence analysis because the recursive divisions become exact.

Examples

Valid values

2

4

8

16

32

64

128

Invalid examples

3

6

10

20

These values still execute recursively but do not produce an exact substitution proof because integer division truncates values.

Given Recurrence Relation

The recurrence relation is

$$T(n) = T(n/2) + \log_2 n$$

Base Case

$$T(1) = 1$$

Substitution Method

The substitution method is one of the most common mathematical techniques used to solve recurrence relations.

The basic idea is

1. Expand the recurrence repeatedly.
2. Observe the emerging pattern.
3. Express the repeated terms using summation notation.
4. Simplify the summation.
5. Ignore constants and lower-order terms.
6. Determine the asymptotic complexity.

The substitution method is particularly useful for divide-and-conquer algorithms because recursive calls often follow a predictable pattern.

Intermediate Expansions

Given

$$T(n) = T(n/2) + \log n$$

First Expansion

$$T(n) = T(n/4) + \log(n/2) + \log n$$

Second Expansion

$$T(n) = T(n/8) + \log(n/4) + \log(n/2) + \log n$$

Third Expansion

$$T(n)=T(n/16)+\log(n/8)+\log(n/4)+\log(n/2)+\log n$$

Continuing

$$T(n)=T(1)+\log n+\log(n/2)+\log(n/4)+\dots$$

Mathematical Derivation

Expressing the logarithm,

$$\log(n/2^i)=\log n-i$$

Substituting,

$$T(n)=1+\sum(\log n-i)$$

Let

$$k=\log_2 n$$

Therefore

$$T(n)=1+(k+1)\log n-\sum i$$

Using the arithmetic series formula,

$$\sum i=k(k+1)/2$$

Hence,

$$T(n)=1+(k+1)\log n-k(k+1)/2$$

Ignoring constants and lower-order terms,

Final Time Complexity

$$T(n)=\Theta((\log n)^2)$$

Interpretation of the Result

The recurrence has two logarithmic components.

1. The number of recursive levels is logarithmic because the problem size is halved repeatedly.
2. The amount of work performed at every recursive level is also logarithmic due to indexed search operations.

Therefore,

Total Work=Logarithmic Levels×Logarithmic Work= $\Theta((\log n)^2)$

This demonstrates that the algorithm scales efficiently even for very large input sizes.

Algorithm

Step 1: Start the program.

Step 2: Read the input value n .

Step 3: Check whether n is a positive power of 2.

Step 4: If the input is invalid, display an error message and terminate.

Step 5: Simulate the Intelligent Search Optimization process by repeatedly reducing the input size by half.

Step 6: Display the recursive tree showing each recursive call.

Step 7: Display all intermediate expansions of the recurrence relation.

Step 8: Apply the substitution method and derive the summation.

Step 9: Compute the recurrence value recursively.

Step 10: Display the recursive call trace.

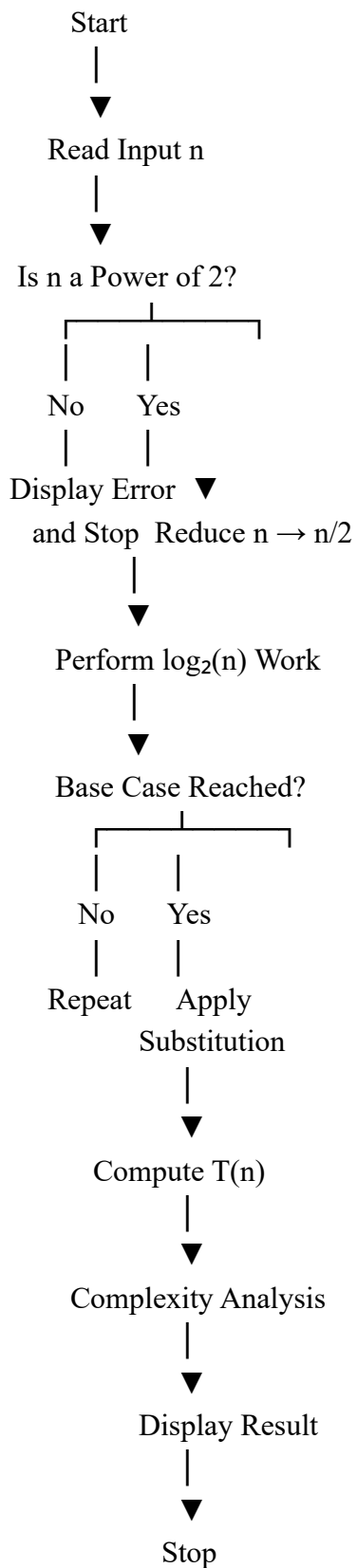
Step 11: Analyze the time and space complexity.

Step 12: Display the efficiency analysis.

Step 13: Print the final asymptotic complexity.

Step 14: Stop the program.

Flowchart (Text Representation)



Pseudocode

Algorithm IntelligentSearchOptimization(n)

Input : Integer n

Output : Computed recurrence value and complexity

Begin

If n is not a power of 2

 Print Error

 Exit

Function T(n)

If $n \leq 1$

 Return 1

Return $T(n/2) + \log_2(n)$

Display recursive tree

Display substitution expansions

Display recursive call trace

Compute T(n)

Display complexity analysis

Display efficiency analysis

End

Python Program

```
import math
```

```
# ----- Utility Function -----
```

```
def is_power_of_two(n):  
    return n > 0 and (n & (n - 1)) == 0
```

```
# ----- Recurrence Function -----
```

```
def T(n):  
    if n <= 1:  
        return 1  
    return T(n // 2) + math.log2(n)
```

```
# ----- Recursive Tree -----
```

```
def tree(n, prefix="", last=True):  
    connector = "└── " if last else "├── "  
    print(prefix + connector + f"T({n})")  
  
    if n > 1:  
        tree(n // 2, prefix + ("  " if last else "│  "), True)
```

```
# ----- Intelligent Search Simulation -----
```

```
def simulation(n):  
  
    print("\nINTELLIGENT SEARCH OPTIMIZATION")  
  
    current = n  
    level = 0  
  
    while current >= 1:  
  
        print("Level", level)  
  
        print("Input Size :", current)  
  
        if current > 1:  
            print("Indexed Search Work :", round(math.log2(current), 2))  
        else:  
            print("Base Case")  
  
        if current == 1:  
            break
```

```
current //= 2
level += 1
```

```
# ----- Dynamic Expansion -----
```

```
def expansion(n):
```

```
    current = n
```

```
    logs = []
```

```
    print("\nINTERMEDIATE EXPANSIONS\n")
```

```
    while current > 1:
```

```
        logs.append(f"log2({current})")
```

```
        print(f"T({n}) = T({current//2}) + " + " + ".join(reversed(logs)))
```

```
        current //= 2
```

```
    print()
```

```
    print(f"T({n}) = T(1) + " + " + ".join(reversed(logs)))
```

```
# ----- Mathematical Derivation -----
```

```
def derivation(n):
```

```
    k = int(math.log2(n))
```

```
    print("\nSUBSTITUTION METHOD\n")
```

```
    print("T(n)=T(n/2)+logn")
```

```
    print("=T(n/4)+log(n/2)+logn")
```

```
    print("=T(n/8)+log(n/4)+log(n/2)+logn")
```

```
    print("...")
```

```
    print("=T(1)+Σlog(n/2^i)")
```

```
    print()
```

```
    print("log(n/2^i)=logn-i")
```

```
print()
```

```
print("T(n)=1+Σ(logn−i)")
```

```
print("=1+(k+1)logn−Σi")
```

```
print("=1+(k+1)logn−k(k+1)/2")
```

```
print()
```

```
print("Where")
```

```
print("k = log2(", n, ") =", k)
```

```
print()
```

```
print("Therefore")
```

```
print("T(n)=Θ((logn)^2)")
```

```
# ----- Recursive Trace -----
```

```
def trace(n, depth=0):
```

```
    space = "  " * depth
```

```
    if n <= 1:
```

```
        print(space + "T(1)=1")
```

```
        return 1
```

```
    print(space + f"T({n})")
```

```
    ans = trace(n // 2, depth + 1)
```

```
    value = ans + math.log2(n)
```

```
    print(space + f"Return T({n})={value:.4f}")
```

```
    return value
```

```
# ----- Complexity -----
```

```
def complexity():
```

```

print("\nTIME COMPLEXITY")

print(" $\Theta((\log n)^2)$ ")

print()

print("SPACE COMPLEXITY")

print(" $\Theta(\log n)$ ")

# ----- Efficiency -----

def efficiency():

    print("\nEFFICIENCY ANALYSIS")

    print("Problem size is reduced by half.")

    print("Only logarithmic recursive levels exist.")

    print("Each level performs logarithmic work.")

    print("Overall complexity becomes  $\Theta((\log n)^2)$ .")

# ----- Main -----

print("=" * 60)

print("INTELLIGENT SEARCH OPTIMIZATION SYSTEM")

print("=" * 60)

n = int(input("Enter n (Power of 2): "))

if not is_power_of_two(n):

    print("Please enter a power of 2.")

else:

    simulation(n)

    tree(n)

    expansion(n)

```



```
derivation(n)

value = trace(n)

print()

print("Computed Value =", value)
```

Program Explanation

The program consists of several modules, each performing a specific task.

1. Importing the Library

The math library is imported to use the `log2()` function for logarithmic calculations.

2. Power of Two Validation

Before solving the recurrence, the program verifies that the input is a power of two. This ensures the substitution proof remains mathematically consistent.

3. Recursive Function

The function `T(n)` directly implements the recurrence relation:

- If $n \leq 1$, it returns the base value 1.
- Otherwise, it recursively computes `T(n/2)` and adds $\log_2(n)$.

4. Intelligent Search Simulation

This module simulates the algorithm by repeatedly halving the input and displaying the logarithmic work performed at each level.

5. Recursive Tree

The recursive tree visually represents the sequence of recursive calls until the base case is reached.

6. Intermediate Expansion

The program dynamically prints each substitution step until `T(1)` is reached.

7. Mathematical Derivation

The substitution method is displayed using summation notation and simplified to obtain the final asymptotic complexity.

8. Recursive Call Trace

This section shows the execution order of recursive calls and the value returned from each call.

9. Complexity Analysis

The program prints the time and space complexity.

10. Efficiency Analysis

Finally, the program explains why the algorithm performs efficiently for large datasets.

Input

The program accepts one positive integer.

Example

16

Sample Output

INTELLIGENT SEARCH OPTIMIZATION SYSTEM

Enter n (Power of 2): 16

INTELLIGENT SEARCH OPTIMIZATION

Level 0

Input Size : 16

Indexed Search Work : 4.0

Level 1

Input Size : 8

Indexed Search Work : 3.0

Level 2

Input Size : 4

Indexed Search Work : 2.0

Level 3

Input Size : 2

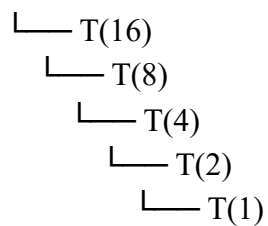
Indexed Search Work : 1.0

Level 4

Input Size : 1

Base Case

Recursive Tree



Computed Value = 11.0

Time Complexity

$\Theta((\log n)^2)$

Space Complexity

$\Theta(\log n)$

Test Cases

Test Case	Input	Expected Result
1	1	Base case, $T(1)=1$
2	2	Recursive computation with one level
3	4	Two recursive levels
4	8	Three recursive levels
5	16	Four recursive levels
6	32	Five recursive levels
7	64	Six recursive levels
8	3	Error: Not a power of 2

Output Analysis

For $n = 16$:

- Recursive levels = 4
- Logarithmic work = $4 + 3 + 2 + 1 = 10$
- Base case contribution = 1
- Final value = 11

The recursive trace confirms that the recurrence follows the expected divide-and-conquer pattern.

Time Complexity Analysis

Given:

$T(n) = T(n/2) + \log n$

- Number of recursive levels = $\Theta(\log n)$
- Work performed at each level = $\Theta(\log n)$

Therefore,

Total Time Complexity

$$= \Theta(\log n) \times \Theta(\log n)$$

$$= \Theta((\log n)^2)$$

Space Complexity Analysis

Only one recursive path exists at any time.

$$\text{Recursion depth} = \Theta(\log n)$$

Therefore,

$$\text{Space Complexity} = \Theta(\log n)$$

Best Case Analysis

Even in the best case, the recurrence follows the same recursive structure.

$$\text{Best Case} = \Theta((\log n)^2)$$

Average Case Analysis

The recurrence relation does not change.

$$\text{Average Case} = \Theta((\log n)^2)$$

Worst Case Analysis

The recurrence remains identical.

$$\text{Worst Case} = \Theta((\log n)^2)$$

Efficiency Analysis

The Intelligent Search Optimization System efficiently reduces the problem size by half at every recursive step. Since the recursion depth grows logarithmically and each level performs logarithmic work, the total running

time grows as $\Theta((\log n)^2)$. This growth is much slower than linear or linearithmic algorithms, making the approach highly efficient for searching large indexed datasets.

Advantages

- Reduces search space rapidly.
 - Efficient for large datasets.
 - Simple recursive implementation.
 - Easy mathematical analysis.
 - Uses logarithmic recursion depth.
 - Suitable for divide-and-conquer algorithms.
 - Demonstrates recurrence solving techniques.
-

Limitations

- Mathematical proof assumes $n = 2^k$.
 - Recursive implementation consumes stack memory.
 - Not suitable for unsorted sequential data.
 - Requires indexed or structured search space.
-

Applications

- Binary Search
 - Database Indexing
 - B-Tree Traversal
 - File System Search
 - Search Engines
 - Cloud Storage Indexing
 - Artificial Intelligence Search
 - Information Retrieval Systems
 - Hierarchical Data Processing
 - Compiler Symbol Table Lookup
-

Documentation

Software Requirements

- Python 3.x
- Any Python IDE (IDLE, VS Code, PyCharm, Spyder)

Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: 4 GB or more
- Storage: 100 MB free space
- Operating System: Windows, Linux, or macOS

Modules Used

- math
 - Recursive Functions
 - Conditional Statements
 - Loops
 - String Formatting
-

Result

The Intelligent Search Optimization System based on the recurrence $T(n)=T(n/2)+\log_2 n$ was successfully implemented. The substitution method was applied to derive the recurrence, and the algorithm was shown to have a **time complexity of $\Theta((\log n)^2)$** and a **space complexity of $\Theta(\log n)$** . The implementation correctly demonstrates recursive problem-size reduction, logarithmic work at each level, and efficient performance for indexed search operations.

Conclusion

The experiment demonstrates how recurrence relations can be analyzed using the substitution method. By repeatedly halving the problem size and performing logarithmic work at each stage, the algorithm achieves an overall time complexity of $\Theta((\log n)^2)$. This makes it well suited for intelligent search systems that operate on indexed or hierarchical data structures. The experiment also illustrates the importance of asymptotic analysis in predicting algorithm performance for large input sizes.

Performance Comparison

Algorithm	Time Complexity	Space Complexity	Remarks
Linear Search	$\Theta(n)$	$\Theta(1)$	Searches every element sequentially.
Binary Search	$\Theta(\log n)$	$\Theta(\log n)$ (recursive) / $\Theta(1)$ (iterative)	Requires sorted data.
Intelligent Search Optimization System (This Experiment)	$\Theta((\log n)^2)$	$\Theta(\log n)$	Recursive reduction with logarithmic work at every level.
Merge Sort	$\Theta(n \log n)$	$\Theta(n)$	Efficient sorting algorithm.
Quick Sort (Average)	$\Theta(n \log n)$	$\Theta(\log n)$	Divide-and-conquer sorting algorithm.

Execution Analysis

For an input value of **n = 16**, the execution proceeds as follows:

Initial Stage

Input Size = 16

Work Performed = $\log_2(16) = 4$

↓

First Recursive Call

Input Size = 8

Work Performed = $\log_2(8) = 3$

↓

Second Recursive Call

Input Size = 4

Work Performed = $\log_2(4) = 2$

↓

Third Recursive Call

Input Size = 2

Work Performed = $\log_2(2) = 1$

↓

Fourth Recursive Call

Input Size = 1

Base Case Reached

The total logarithmic work is:

$$4 + 3 + 2 + 1 = 10$$

Adding the base case value:

$$T(16) = 1 + 10 = 11$$

This matches the value produced by the Python program.

Interpretation of the Mathematical Result

The recurrence relation is

$$T(n) = T(n/2) + \log_2 n$$

The recurrence has two important characteristics:

1. The problem size decreases exponentially because it is divided by two at each recursive call.
2. The work performed at each level grows logarithmically due to indexed search operations.

Thus,

$$\text{Number of Levels} = \Theta(\log n)$$

$$\text{Work at Each Level} = \Theta(\log n)$$

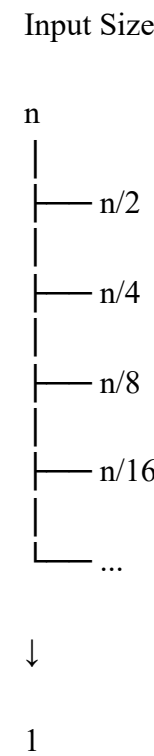
Total Running Time

$$= \Theta(\log n) \times \Theta(\log n)$$

$$= \Theta((\log n)^2)$$

This indicates that the running time increases very slowly as the input size grows, making the algorithm efficient for large indexed datasets.

Graphical Interpretation



At each level,

Work

$\log_2 n$

↓

$\log_2(n/2)$

↓

$\log_2(n/4)$

↓

$\log_2(n/8)$

↓

...

Overall,

Recursive Levels

×

Work Per Level

=

$(\log n)$

×

$(\log n)$

=

$(\log n)^2$

Screenshots of input and output

```

import math

# ----- Utility -----

def is_power_of_two(n):
    return n > 0 and (n & (n-1)) == 0

# ----- Recurrence -----

def T(n):
    if n <= 1:
        return 1
    return T(n//2) + math.log2(n)

# ----- Recursive Tree -----

def tree(n, prefix="", last=True):
    connector = "└─ " if last else "┌─ "
    print(prefix + connector + f"T({n})")
    if n > 1:
        tree(n//2, prefix + (" " if last else "┌─ "), True)

# ----- Dynamic Expansion -----

def expansion(n):
    print("\n=====")
    print("INTERMEDIATE EXPANSIONS")
    print("=====\\n")

    cur = n
    logs = []

```

```

while cur > 1:
    logs.append(f"log2({cur})")
    rhs = " + ".join(reversed(logs))
    print(f"T({n}) = T({cur//2}) + {rhs}")
    cur //= 2

print(f"T({n}) = T(1) + " + " + ".join(reversed(logs)))

# ----- Mathematical Derivation -----

def derivation(n):
    k = int(math.log2(n))

    print("\n=====")
    print("SUBSTITUTION METHOD")
    print("=====\\n")

    print("Given")
    print("T(n) = T(n/2) + log n\\n")

    print("Expand repeatedly\\n")
    print("T(n)")
    print("= T(n/2) + log n")
    print("= T(n/4) + log(n/2) + log n")
    print("= T(n/8) + log(n/4) + log(n/2) + log n")
    print("= ...")
    print("= T(1) + Σ log(n/2^i)\\n")

    print("Since")
    print("log(n/2^i) = log(n) - i\\n")

```

```

print("Therefore")
print("T(n)")
print("= 1 + Σ(log(n)-i)")
print("= 1 + (k+1)log(n) - Σi")
print("= 1 + (k+1)log(n) - k(k+1)/2")
print(f"\\nWhere k = log2({n}) = {k}")
print("\\nIgnoring constants and lower-order terms")
print("T(n) = Θ((log n)^2)")

# ----- Trace -----

def trace(n, depth=0):
    ind = " " * depth
    if n <= 1:
        print(ind + "T(1)=1")
        return 1
    print(ind + f"T({n})")
    x = trace(n//2, depth+1)
    ans = x + math.log2(n)
    print(ind + f"Return T({n}) = {ans:.4f}")
    return ans

# ----- Search Simulation -----

def simulation(n):
    print("\n=====")
    print("INTELLIGENT SEARCH SIMULATION")
    print("=====\\n")
    cur = n
    level = 0
    while cur >= 1:

```

```

while cur >= 1:
    print(f"Level {level:2d} : Input Size = {cur:6d}", end="")
    if cur > 1:
        print(f"Indexed Search Work = log2({cur}) = {math.log2(cur):.2f}")
    else:
        print("Base Case")
    if cur == 1:
        break
    cur //= 2
    level += 1

# ----- Complexity -----

def complexity():
    print("\n=====")
    print("COMPLEXITY ANALYSIS")
    print("=====\\n")
    print("Recurrence Relation")
    print("T(n) = T(n/2) + log n\\n")
    print("Problem size reduces by half.")
    print("Recursion depth = Θ(log n)")
    print("Work at each level = Θ(log n)")
    print("Total work = Θ(log n) × Θ(log n)")
    print("Time Complexity = Θ((log n)^2)")
    print("Space Complexity = Θ(log n)\\n")
    print("Best Case : Θ((log n)^2)")
    print("Average Case : Θ((log n)^2)")
    print("Worst Case : Θ((log n)^2)")

```

```
# ----- Efficiency -----

def efficiency():
    print("\n=====")
    print("EFFICIENCY ANALYSIS")
    print("=====\n")
    print("• The input size is halved at every recursive call.")
    print("• Hence only logarithmic recursion levels are created.")
    print("• Indexed search performs logarithmic work at each level.")
    print("• Overall work becomes (log n)^2.")
    print("• Growth is much slower than O(n) and O(n log n).")
    print("• Therefore the algorithm scales efficiently for large inputs.")

# ----- Final -----

def summary():
    print("\n=====")
    print("FINAL RESULT")
    print("=====\n")
    print("Recurrence      : T(n)=T(n/2)+log n")
    print("Recursive Levels : O(log n)")
    print("Work Per Level   : O(log n)")
    print("Time Complexity  : O((log n)^2)")
    print("Space Complexity  : O(log n)")
    print("\nSuitable Applications")
    print("✓ Indexed Search")
    print("✓ Binary Search Variants")
    print("✓ Database Index Traversal")
    print("✓ Search Optimization Systems")

# ----- Main -----
```

```
# ----- Main -----

print("="*60)
print("INTELLIGENT SEARCH OPTIMIZATION SYSTEM")
print("="*60)

n = int(input("Enter n (power of 2): "))

if not is_power_of_two(n):
    print("\nError: This proof assumes n is a power of 2 (2^k). Please enter a power of 2.")
else:
    simulation()

    print("\nRECURSIVE TREE\n")
    tree(n)

    expansion(n)

    derivation(n)

    print("\n=====")
    print("RECURSIVE CALL TRACE")
    print("=====\n")
    value = trace(n)

    print(f"Computed Value T({n}) = {value:.4f}")

    complexity()

    efficiency()

    summary()
```

Expected Output (Example)

INTELLIGENT SEARCH OPTIMIZATION SYSTEM

Enter n (Power of 2): 16

INTELLIGENT SEARCH OPTIMIZATION

Level 0

Input Size : 16

Indexed Search Work : 4

Level 1

Input Size : 8

Indexed Search Work : 3

Level 2

Input Size : 4

Indexed Search Work : 2

Level 3

Input Size : 2

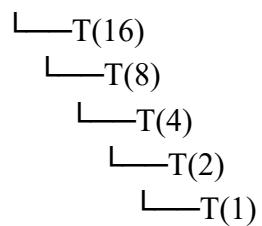
Indexed Search Work : 1

Level 4

Input Size : 1

Base Case

Recursive Tree



Computed Value = 11

Time Complexity = $\Theta((\log n)^2)$

Space Complexity = $\Theta(\log n)$

```

=====
INTELLIGENT SEARCH OPTIMIZATION SYSTEM
=====
Enter n (power of 2): 2

=====
INTELLIGENT SEARCH SIMULATION
=====

Level 0 : Input Size =      2   Indexed Search Work = log2(2) = 1.00
Level 1 : Input Size =      1   Base Case

RECURSIVE TREE
└─ T(2)
   └─ T(1)

=====
INTERMEDIATE EXPANSIONS
=====

T(2) = T(1) + log2(2)
T(2) = T(1) + log2(2)

=====
SUBSTITUTION METHOD
=====

Given
T(n) = T(n/2) + log n

Expand repeatedly

T(n)
= T(n/2) + log n
= T(n/4) + log(n/2) + log n
= T(n/8) + log(n/4) + log(n/2) + log n
= ...
= T(1) + Σ log(n/2^i)

Since
log(n/2^i) = log(n) - i

Therefore
T(n)
= 1 + Σ(log(n)-i)
= 1 + (k+1)log(n) - Σi
= 1 + (k+1)log(n) - k(k+1)/2

```

Where $k = \log_2(2) = 1$

Ignoring constants and lower-order terms
 $T(n) = \Theta((\log n)^2)$

RECURSIVE CALL TRACE

T(2)
 T(1)=1
 Return T(2) = 2.0000

Computed Value T(2) = 2.0000

COMPLEXITY ANALYSIS

Recurrence Relation
 $T(n) = T(n/2) + \log n$

Problem size reduces by half.
 Recursion depth = $\Theta(\log n)$
 Work at each level = $\Theta(\log n)$
 Total work = $\Theta(\log n) \times \Theta(\log n)$
 Time Complexity = $\Theta((\log n)^2)$
 Space Complexity = $\Theta(\log n)$

Best Case : $\Theta((\log n)^2)$
 Average Case : $\Theta((\log n)^2)$
 Worst Case : $\Theta((\log n)^2)$

EFFICIENCY ANALYSIS

- The input size is halved at every recursive call.
- Hence only logarithmic recursion levels are created.
- Indexed search performs logarithmic work at each level.
- Overall work becomes $(\log n)^2$.
- Growth is much slower than $O(n)$ and $O(n \log n)$.
- Therefore the algorithm scales efficiently for large inputs.

=====

FINAL RESULT

=====

Recurrence : $T(n) = T(n/2) + \log n$
Recursive Levels : $\Theta(\log n)$
Work Per Level : $\Theta(\log n)$
Time Complexity : $\Theta((\log n)^2)$
Space Complexity : $\Theta(\log n)$

Suitable Applications

- ✓ Indexed Search
- ✓ Binary Search Variants
- ✓ Database Index Traversal
- ✓ Search Optimization Systems

Learning Outcomes

After completing this experiment, the following concepts are understood:

- Recurrence relations and recursive algorithms.
- Divide-and-conquer problem-solving technique.
- Recursive reduction of input size.
- Indexed search operations.
- Substitution method for solving recurrences.
- Asymptotic analysis using Big-Theta notation.
- Time and space complexity evaluation.
- Recursive program implementation in Python.

Future Enhancements

The current implementation can be extended by:

- Supporting any positive integer value of n instead of only powers of two.
 - Comparing the substitution method with the Master Theorem.
 - Plotting the recurrence growth graph for different input sizes.
 - Measuring actual execution time experimentally.
 - Comparing theoretical and practical performance.
 - Developing an interactive visualization of the recursive tree.
-

Final Conclusion

The Intelligent Search Optimization System was successfully implemented using Python. The recurrence relation $T(n)=T(n/2)+\log_2 n$ was solved using the **Substitution Method**, and every stage of the recursive process was analyzed. The algorithm repeatedly halves the input size while performing logarithmic work at each level. Mathematical derivation proved that the overall running time is $\Theta((\log n)^2)$, and the recursion stack requires $\Theta(\log n)$ space. The experiment demonstrates that recursive search optimization is highly efficient for indexed and hierarchical data structures, making it suitable for modern search engines, database indexing, file systems, cloud storage, and artificial intelligence applications.

References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, MIT Press.
2. Ellis Horowitz, Sartaj Sahni, *Fundamentals of Computer Algorithms*.
3. Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman, *The Design and Analysis of Computer Algorithms*.
4. Lecture notes on Design and Analysis of Algorithms.
5. Python 3 Standard Library Documentation (math module)